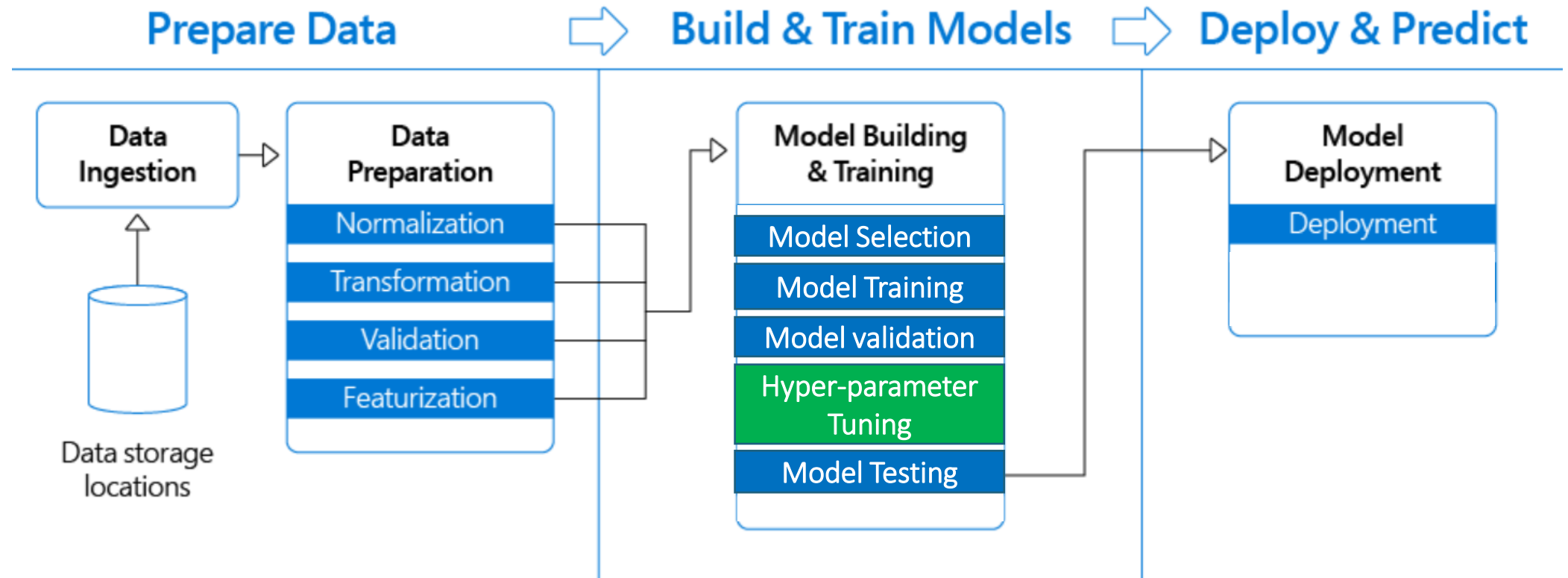




Hyperparameter Tuning Techniques

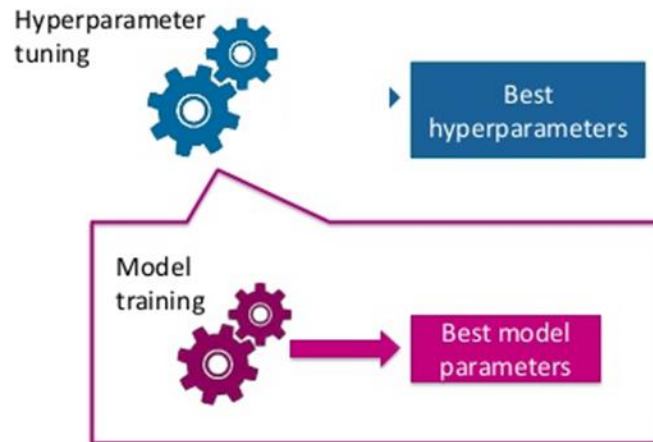
Searching for best parameters
(Scikit-Learn)



Model Generation

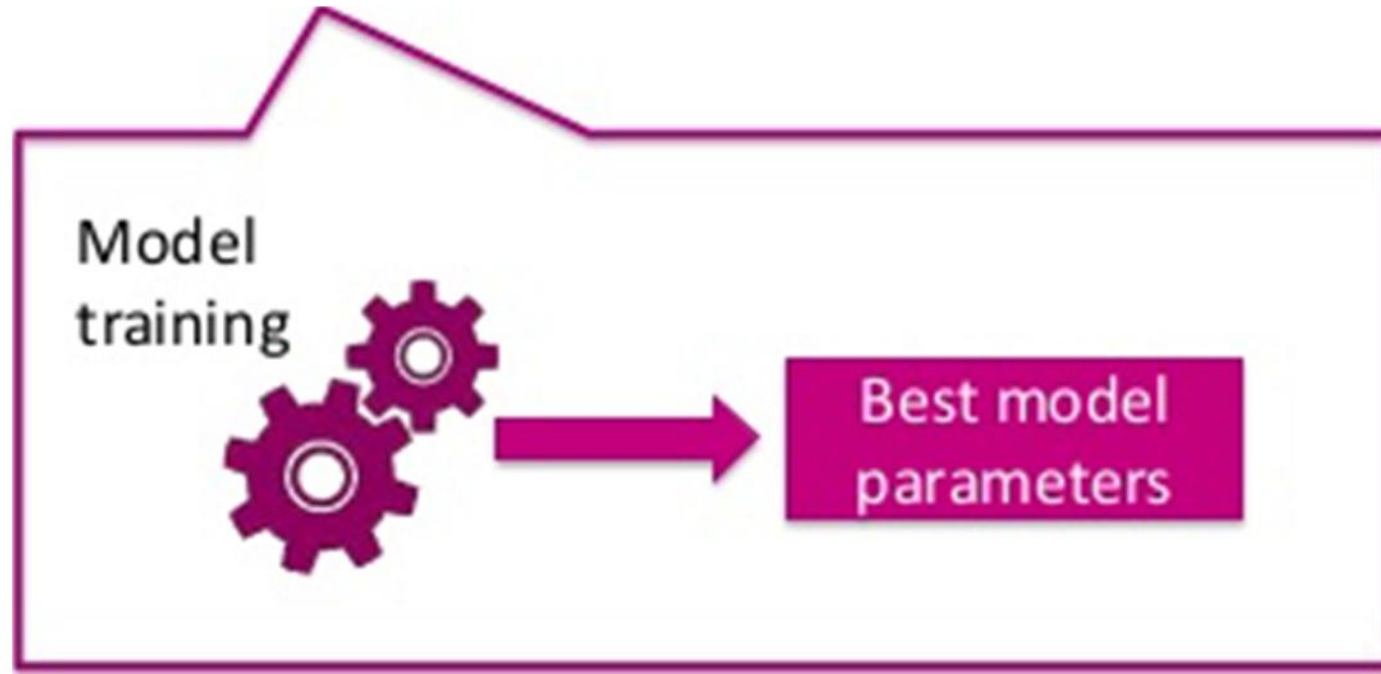
Process from data to model deployment

Hyperparameter tuning vs. model training

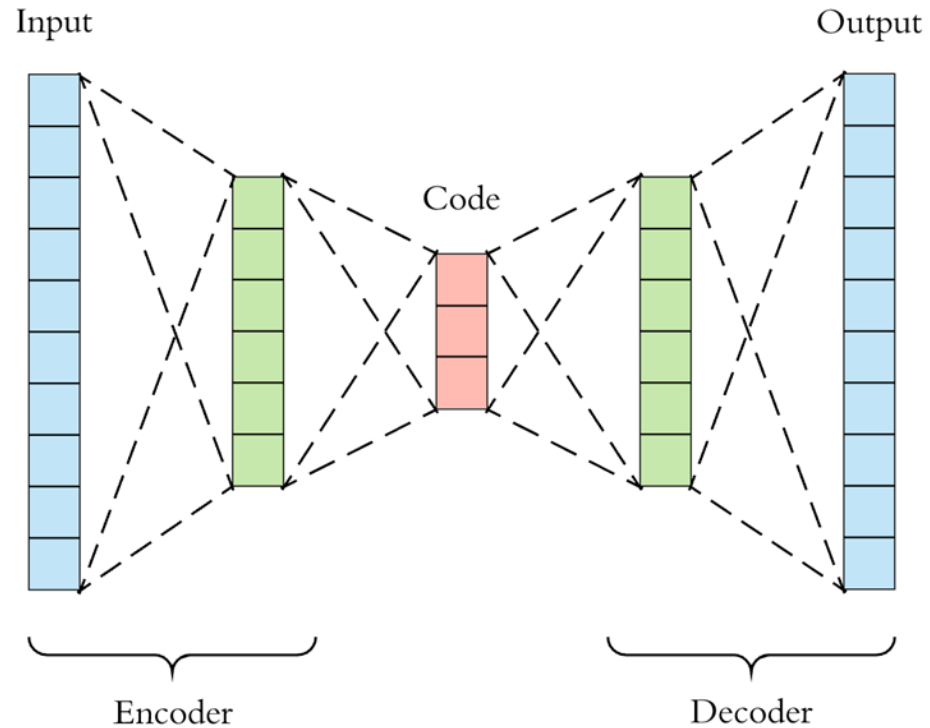


A Comparison

- **Best Model Parameters** (θ) with given tuning parameters
- **Best Hyperparameters** (all tuning parameters)
 - Architecture parameters
 - Layers / Nodes/ kernel types / kernel parameters
 - Optimizer parameters
 - Optimizer / learning rate schedule / Loss function/ constraint parameters
 - Cross validation parameters
 - K-fold / data partition techniques
 - Training parameters
 - Batch_size / epoch



Model Parameters



Model Parameters

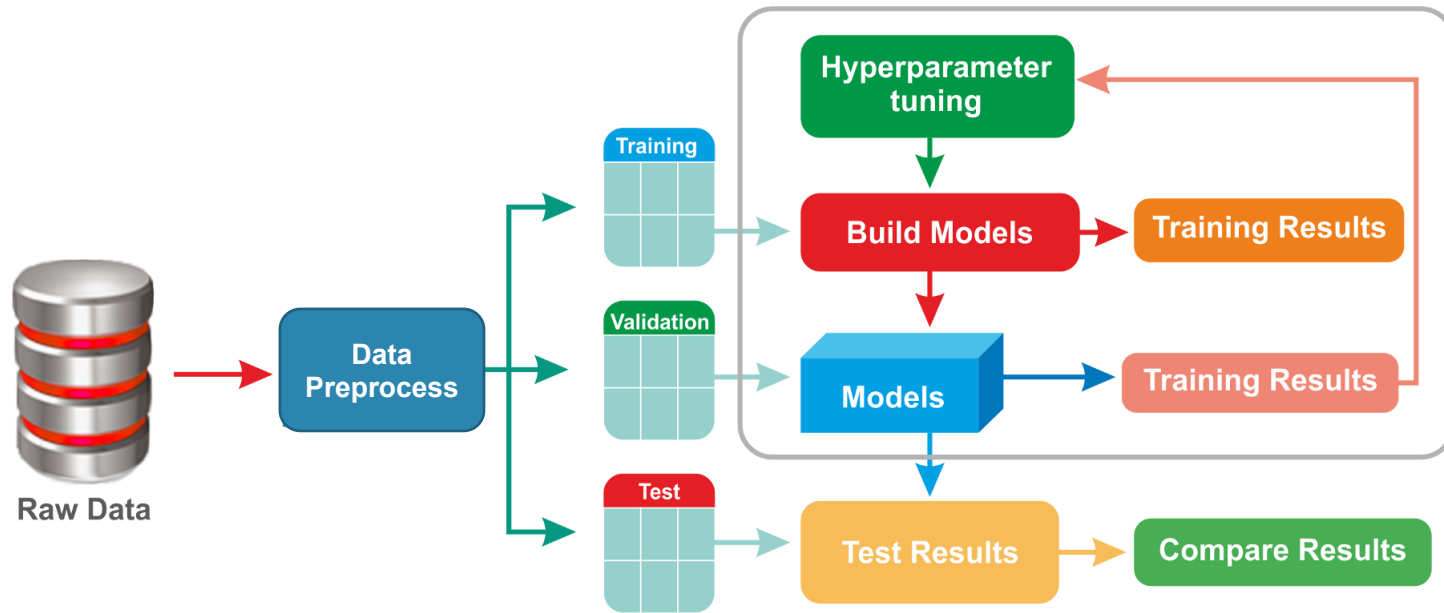
$$y = h_{\theta}(x)$$

$$y = h_{\theta}(x_1, x_2, \dots, x_N)$$

- θ_i
 - $\rightarrow \theta_1 \leq \theta_i \leq \theta_N$ (*model weights*)
 - $\rightarrow \theta_0$ (*Model bias*)
- Optimize θ_i to get best model parameters
 - using predefined hyperparameters

Hyperparameters Tuning

Searching for best hyperparameters



(Model Design + Hyperparameters) → Model Parameters



Hyperparameter Tuning Techniques

Grid Search / Random Search / Hierarchical Search / Smart Search



Hyperparameter Tuning

- **Iterative changing** hyperplane parameters
 - Best fit for **validation / testing**
- **Techniques**
 - User Trial & Error (manual selection)
 - Grid Search
 - Random Search
 - Hierarchical Search
 - RandomSearch -> Scope good parameters
 - Grid Search -> around good parameters
 - Smart Search
 - Gaussian process / Bayesian Optimization
 - Random Forest
 - Genetic Algorithm

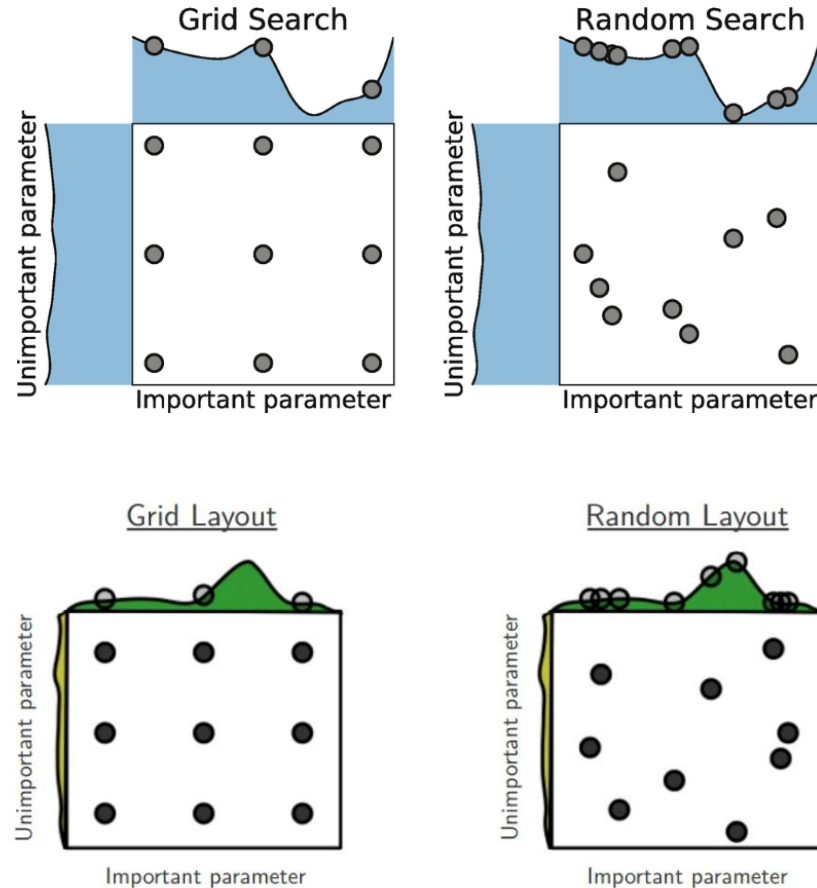


Image from <http://jmlr.csail.mit.edu/papers/volume13/bergstra12a/bergstra12a.pdf>



Hyperparameter Tuning Techniques

Grid Search / Random Search / Hierarchical Search / Smart Search



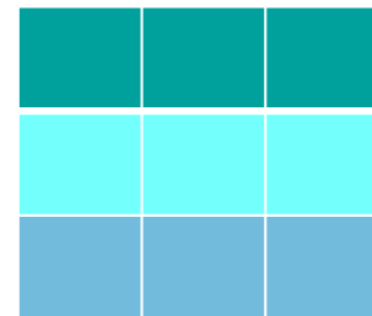
- Hyperparameter Tuning Techniques

- User Trial & Error (manual selection)
- Grid Search
- Random Search
- Hierarchical Search
 - RandomSearch -> Scope good parameters
 - Grid Search -> around good parameters

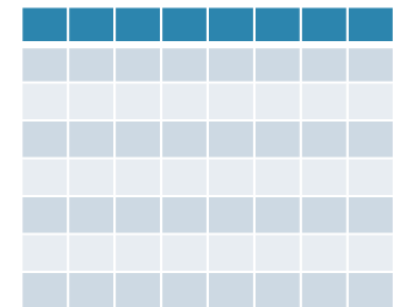
- Full combination of parameter list

- Accuracy depends on

- ความละเอียดของ grid
 - Coarse Grid (Grid หยาบ / ห่าง)
 - Fine Grid (Grid ละเอียด)



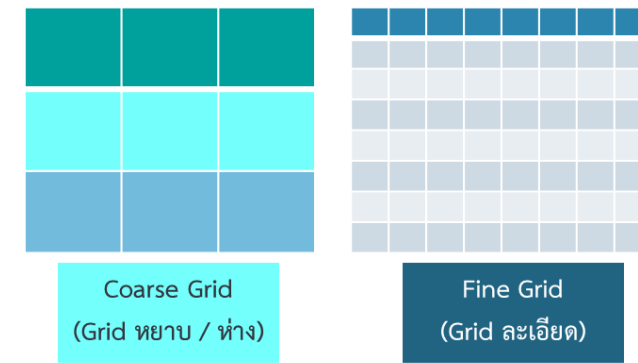
Coarse Grid
(Grid หยาบ / ห่าง)



Fine Grid
(Grid ละเอียด)

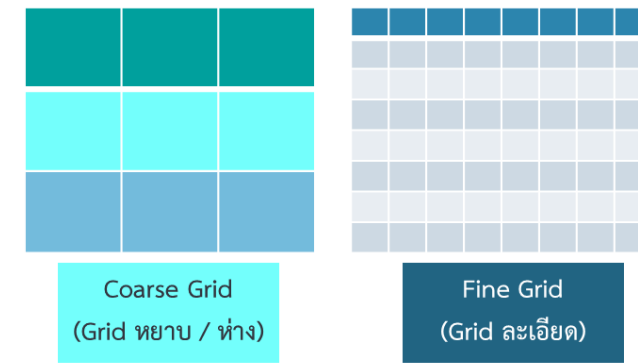
- Hyperparameter Tuning Techniques

- User Trial & Error (manual selection)
- **Grid Search**
- Random Search
- Hierarchical Search
 - RandomSearch -> Scope good parameters
 - Grid Search -> around good parameters



- Full combination of parameter list
- Accuracy depends on
 - ความละเอียดของ grid
 - **Coarse Grid** (Grid หยาบ / ห่าง)
 - จำนวนพารามิเตอร์น้อย
 - เวลาในการประมวลผลน้อย
 - Processing resource น้อย
 - มีโอกาสสูง
 - ข้ามพารามิเตอร์ที่ดีที่สุดไป
 - Fine Grid (Grid ละเอียด)

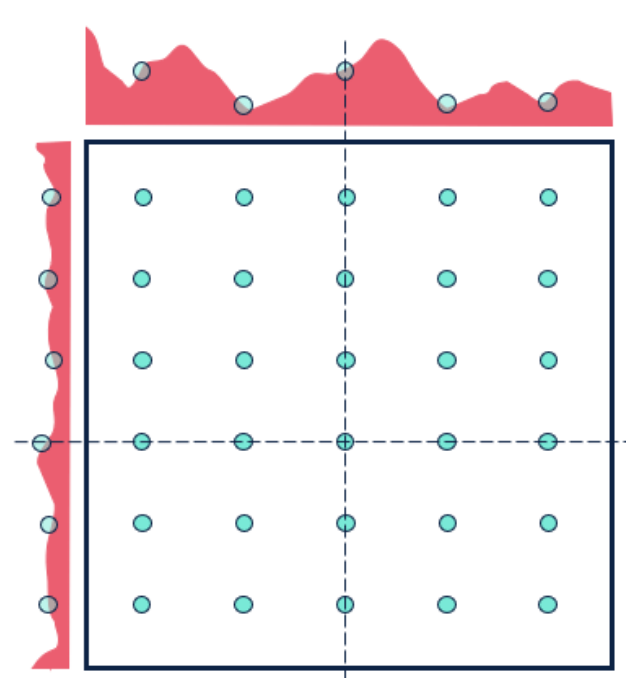
- Hyperparameter Tuning Techniques
 - User Trial & Error (manual selection)
 - **Grid Search**
 - Random Search
 - Hierarchical Search
 - RandomSearch -> Scope good parameters
 - Grid Search -> around good parameters



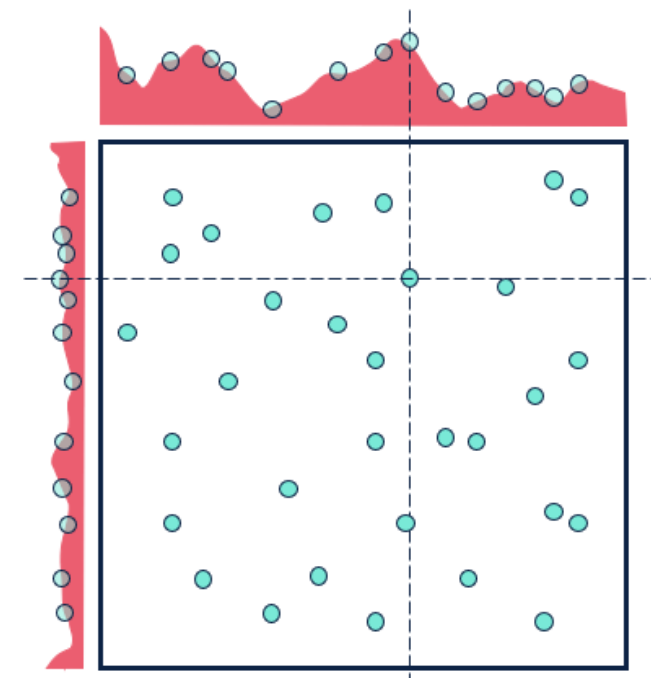
- Full combination of parameter list
- Accuracy depends on
 - ความละเอียดของ grid
 - Coarse Grid (Grid หยาบ / ห่าง)
 - **Fine Grid** (Grid ละเอียด)
 - มีโอกาส สูง
 - พบตำแหน่งพารามิเตอร์ที่ดีที่สุดไป
 - จำนวนพารามิเตอร์ มาก
 - เวลาในการประมวลผล สูงมาก
 - Processing resource สูงมาก

Parameter Search Space Comparison

(Grid Search vs Random Search with Same number of parameter combination)



Grid Search



Random Search



Hyperparameter Tuning Techniques

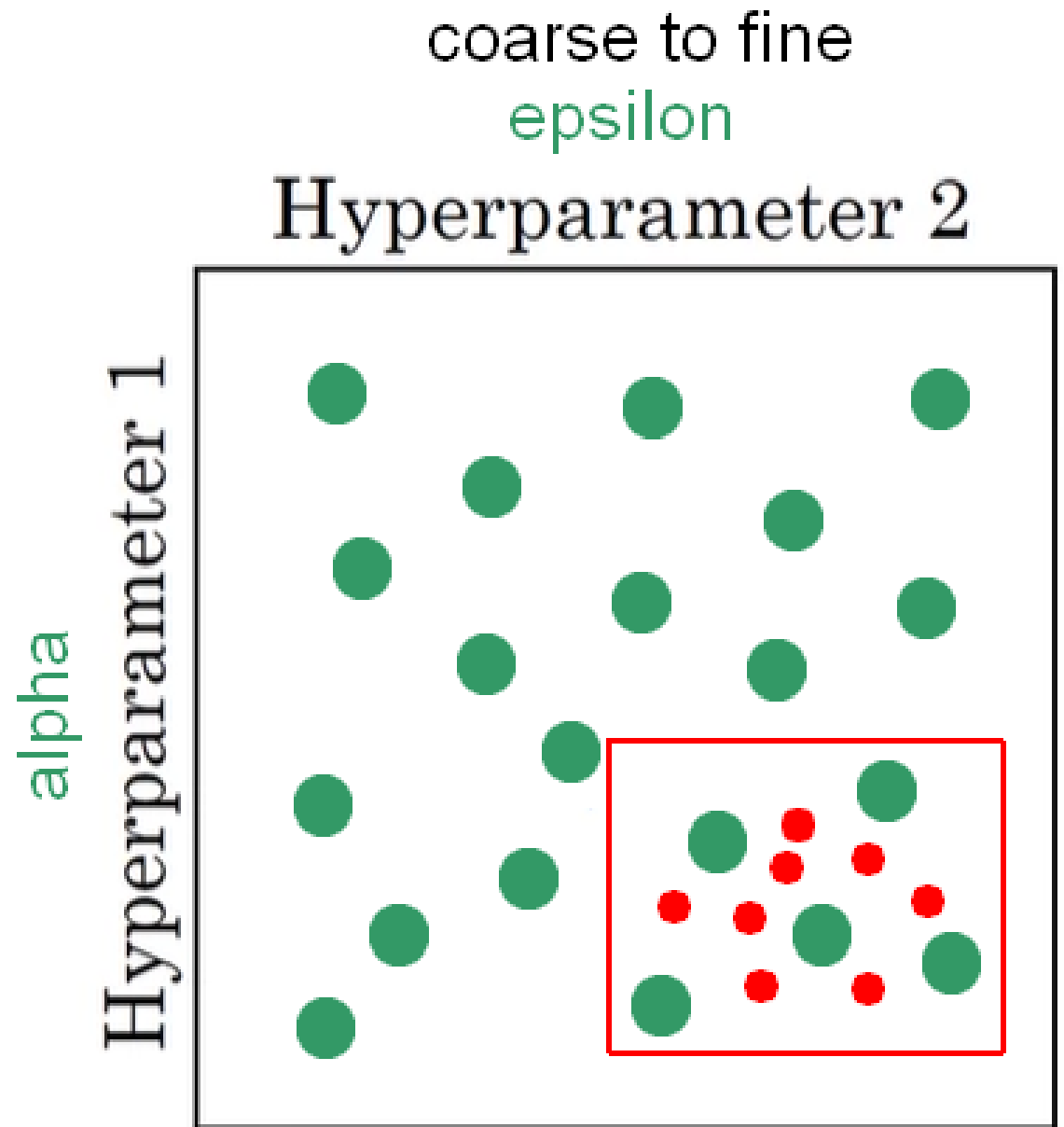
Grid Search / Random Search / **Hierarchical Search** / Smart Search



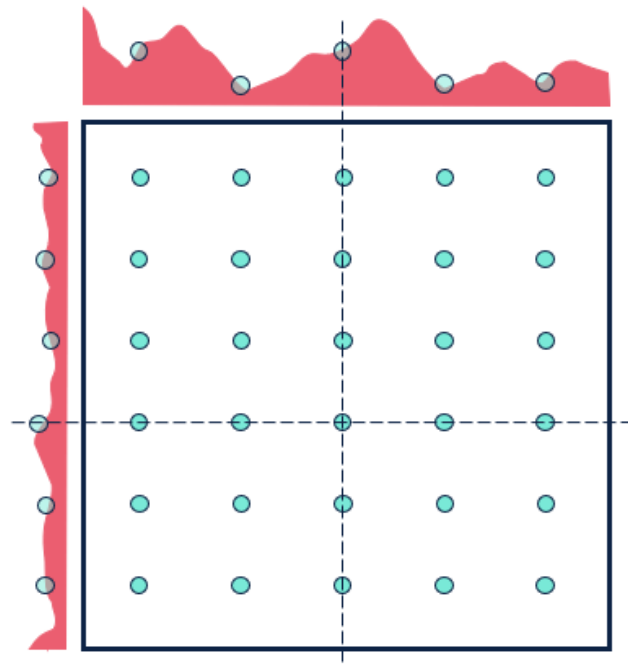
Hierarchical Search

Multi-level search

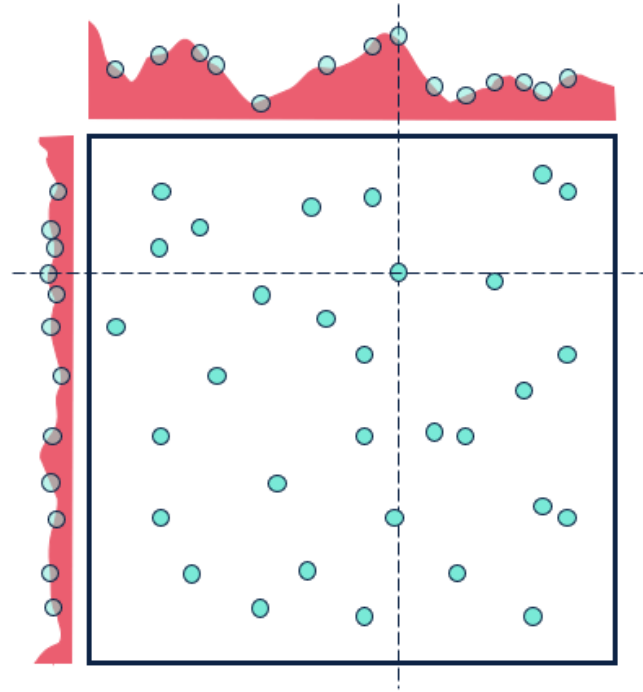
From coarse to fine



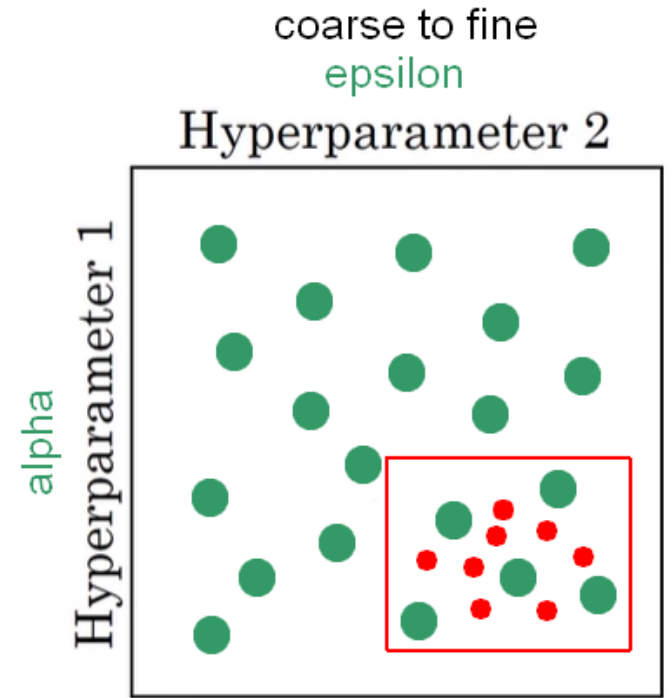
Today's Topic



Grid Search



Random Search





Hyperparameter Tuning Techniques

Raytune



Raytune

a table comparing **grid search** in Ray Tune with other popular libraries:

Library	Grid Search Implementation	Strengths	Weaknesses	Framework Support
Ray Tune	<code>tune.grid_search</code> function	Easy to use, integrates with Tune's distributed training and other search algorithms	Can be computationally expensive for a large number of hyperparameters	TensorFlow, PyTorch , XGBoost, LightGBM, scikit-learn, Keras
Scikit-learn GridSearchCV	<code>sklearn.model_selection.GridSearchCV</code>	Simple to use, supports various scoring metrics	Limited to scikit-learn models, doesn't offer distributed execution	Scikit-learn only Numpy array based
Optuna	<code>optuna.samplers.GridSampler</code> or custom	Flexible, supports defining custom grids	Requires more code compared to Ray Tune's built-in grid search	Can work with various frameworks (custom implementation needed)
Hyperopt	<code>hyperopt.hp.grid</code> function	Supports early stopping and advanced features	Can be more complex to use compared to Ray Tune's grid search	Can work with various frameworks (custom implementation needed)

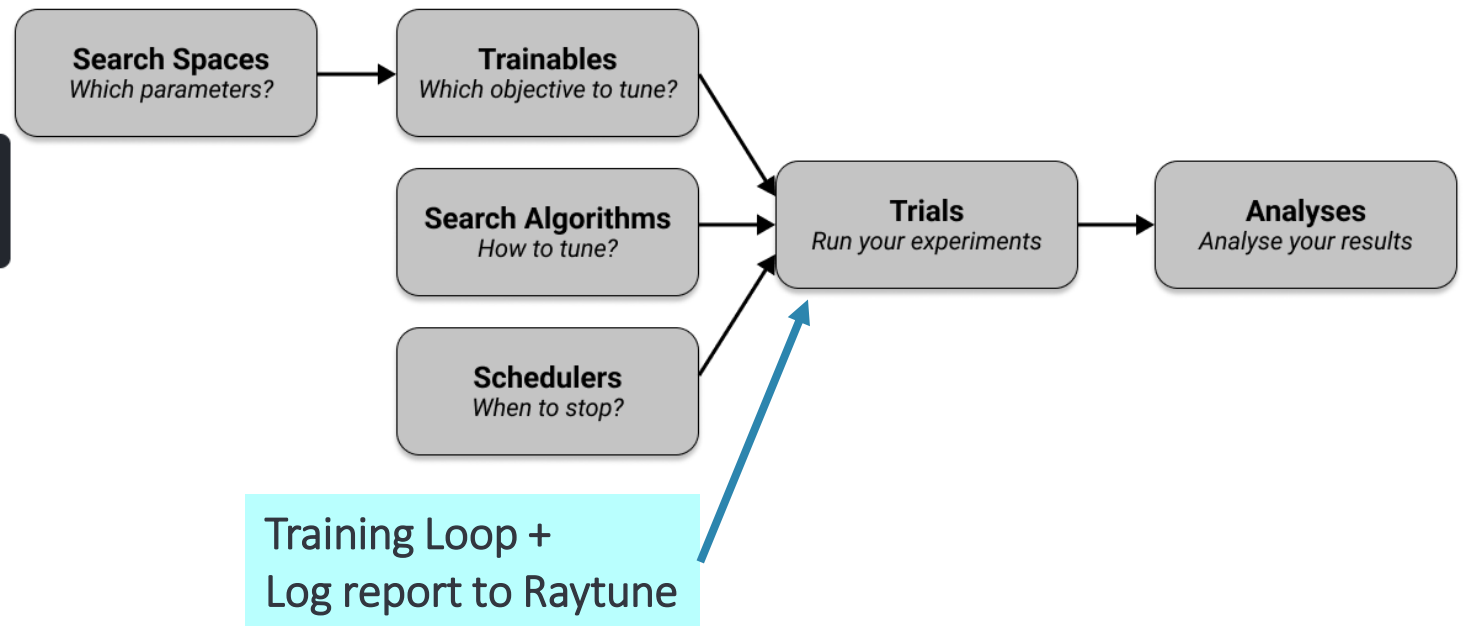
Raytune

1. Initialize Ray

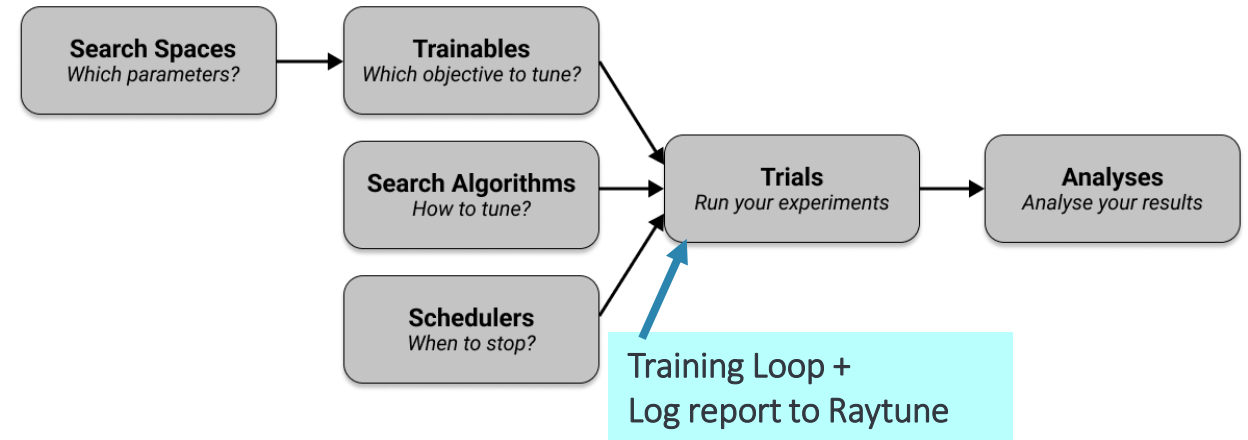
```
1 ray.init(num_gpus=1,)
```

2. define the search space
3. define the trainable functions
4. specify the search algorithm,
5. set up the scheduler and resources
6. Trials
7. Analyses

- Resource :
- - [\[A Guide To Parallelism and Resources for Ray Tune\]](#)
- - [\[Working with Tune Search Spaces\]](#)
- - [\[How to configure logging in Tune?\]](#)
- - [\[Tune Trial Schedulers \(`tune.schedulers`\)\]](#)



Raytune



1. Initialize Ray

2. define the search space:

A search space defines valid values for your hyperparameters and can specify how these values are sampled (e.g. from a uniform distribution or a normal distribution).

3. define the trainable functions

4. specify the search algorithm,

5. set up the scheduler and resources

6. Trials

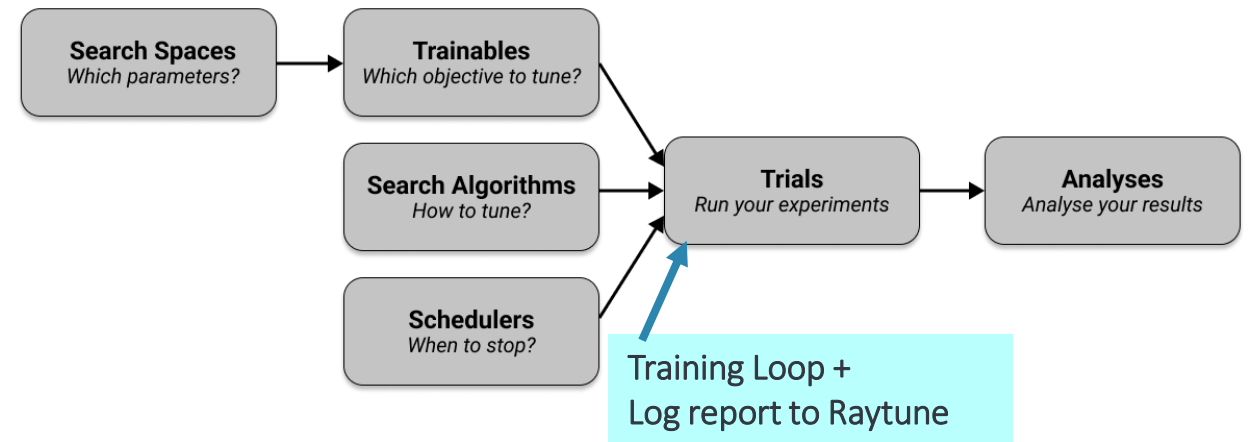
7. Analyses

```
# Define the search space for hyperparameters
search_space = {
    "learning_rate": tune.grid_search([0.001, 0.01]), # Adjust learning
    "num_filters": tune.grid_search([32, 64]) # Adjust number of filter
}
```

```
config = {
    'model_name': tune.grid_search(models_ls),
    'h_dims': tune.grid_search(h_dims),
    'optimizer': tune.grid_search(opts),
    'lr': tune.grid_search(lr),
    'batch_size': tune.grid_search(batch_size),
    'num_epochs': tune.grid_search(num_epochs),
    # 'num_epochs': tune.choice([1])
}
```

Grid search parameters

Raytune



1. Initialize Ray

A search space defines valid values for your hyperparameters and can specify how these values are sampled (e.g. from a uniform distribution or a normal distribution).

2. define the search space:

Tune offers various functions to define search spaces and sampling methods.

3. define the trainable functions

4. specify the search algorithm,

5. set up the scheduler and resources

6. Trials

7. Analyses

```
config = {
    "uniform": tune.uniform(-5, -1), # Uniform float between -5 and -1
    "quniform": tune.quniform(3.2, 5.4, 0.2), # Round to multiples of 0.2
    "loguniform": tune.loguniform(1e-4, 1e-1), # Uniform float in log space
    "qloguniform": tune.qloguniform(1e-4, 1e-1, 5e-5), # Round to multiples of 5e-5
    "randn": tune.randn(10, 2), # Normal distribution with mean 10 and std 2
    "qrandn": tune.qrandn(10, 2, 0.2), # Round to multiples of 0.2
    "randint": tune.randint(-9, 15), # Random integer between -9 and 15
    "qrandint": tune.qrandint(-21, 12, 3), # Round to multiples of 3 (in range)
    "lograndint": tune.lograndint(1, 10), # Random integer in log space
    "qlograndint": tune.qlograndint(1, 10, 2), # Round to multiples of 2
    "choice": tune.choice(["a", "b", "c"]), # Choose one of these options
    "func": tune.sample_from(
        lambda spec: spec.config.uniform * 0.01
    ), # Depends on other value
    "grid": tune.grid_search([32, 64, 128]), # Search over all these values
}
```

Random search parameters

Raytune

1. Initialize Ray
2. define the search space:
3. define the trainable functions with Log report to Raytune
4. specify the search algorithm,
5. set up the scheduler and resources
6. Trials
7. Analyses

```
1 def train_raytune(config):
2     torch.manual_seed(4912)
3
4     # Prepare Data
5
6     # Create Dataloader
7     trainloader = DataLoader(train_dataset, batch_size=config['batch_size'], shuffle=True, num_workers=16)
8     testloader = DataLoader(test_dataset, batch_size=config['batch_size'], shuffle=True, num_workers=16)
9
10
11     device = "cuda" if torch.cuda.is_available() else "cpu"
12     model = Autoencoder()
13     model = model.to(device)
14     loss_fn = nn.MSELoss()
15
16
17     if config['optimizer'] == 'Adam':
18         opt = torch.optim.Adam(model.parameters(), lr=config['lr'])
19     elif config['optimizer'] == 'SGD':
20         opt = torch.optim.SGD(model.parameters(), lr=config['lr'])
```

Model Architecture
+ Optimizer

Raytune

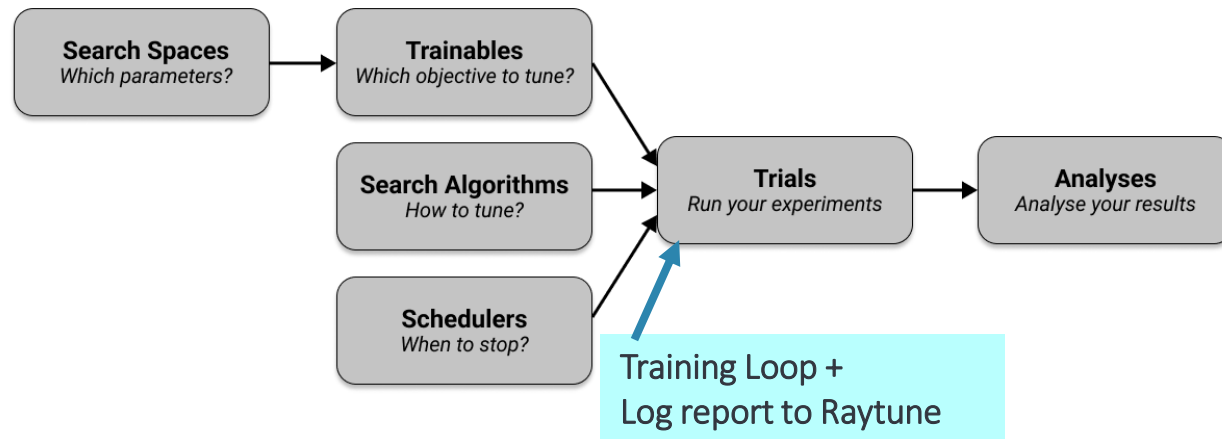
1. Initialize Ray
2. define the search space:
3. define the trainable functions with Log report to Raytune
4. specify the search algorithm,
5. set up the scheduler and resources
6. Trials
7. Analyses

Training Loop

```
1  def train_raytune(config):
2
3      ...
4
5      for epoch in range(config['num_epochs']):
6          model.train()
7          avg_train_loss = 0
8          avg_test_loss = 0
9          for images, gt in trainloader:
10              # Train Model
11
12              avg_train_loss += len(trainloader)
13
14              total_psnr = 0
15              total_ssim = 0
16              model.eval()
17              for images, gt in testloader:
18                  # Evaluate model
19
20              avg_psnr = total_psnr / len(testloader)
21              avg_ssim = total_ssim / len(testloader)
22              avg_test_loss += len(testloader)
23
24              session.report({
25                  "train_loss": avg_train_loss,
26                  "val_loss": avg_test_loss,
27                  "val_psnr": avg_psnr,
28                  "val_ssim": avg_ssim,
29              })
```

Log report to Raytune

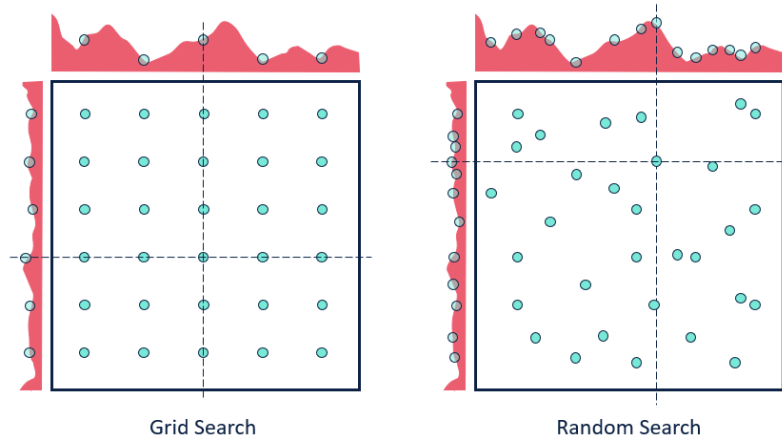
Raytune



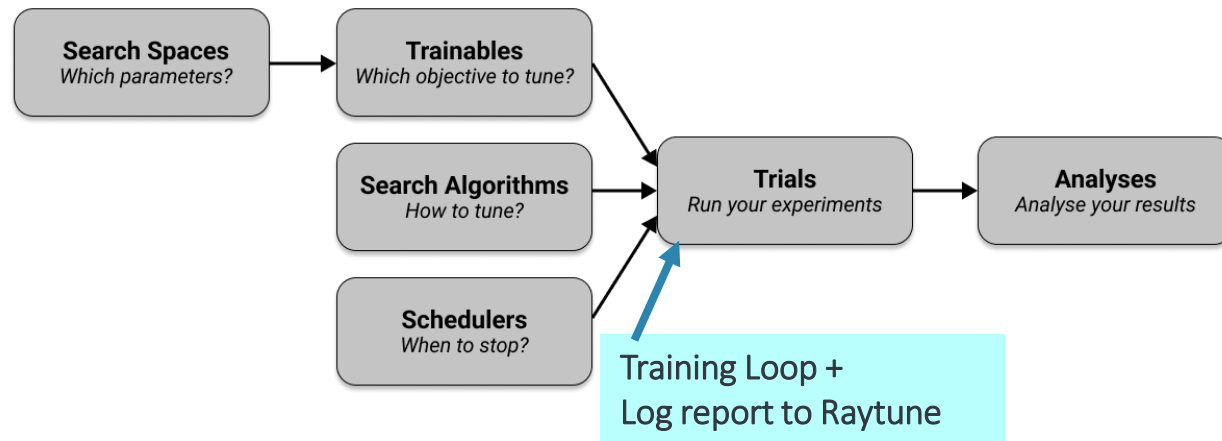
1. Initialize Ray
2. define the search space:
3. define the trainable functions
4. **specify the search algorithm**
5. set up the scheduler and resources
6. **Trials**
7. Analyses

You use `Tuner.fit()` to execute and manage hyperparameter tuning and generate your trials. At a minimum, your Tuner call takes in model and a **trainable** as first argument, and a **param_space** dictionary to define the search space.

To optimize the hyperparameters of your training process, you use a [Search Algorithm](#) which suggests hyperparameter configurations. If you don't specify a search algorithm, Tune will use **random search by default**, which can provide you with a good starting point for your hyperparameter optimization.



Raytune



To optimize the hyperparameters of your training process, you use a [Search Algorithm](#) which suggests hyperparameter configurations. If you don't specify a search algorithm, Tune will use **random search by default**, which can provide you with a good starting point for your hyperparameter optimization.

1. Initialize Ray
2. define the search space:
3. define the trainable functions
4. **specify the search algorithm**
5. set up the scheduler and resources
6. Trials
7. Analyses

```
class BasicVariantGenerator(SearchAlgorithm):
    """Uses Tune's variant generation for resolving variables.

    This is the default search algorithm used if no other search algorithm
    is specified.

    Args:
        points_to_evaluate: Initial parameter suggestions to be run
            first. This is for when you already have some good parameters
            you want to run first to help the algorithm make better suggestions
            for future parameters. Needs to be a list of dicts containing the
            configurations.
        max_concurrent: Maximum number of concurrently running trials.
            If 0 (default), no maximum is enforced.
        constant_grid_search: If this is set to ``True``, Ray Tune will
            *first* try to sample random values and keep them constant over
            grid search parameters. If this is set to ``False`` (default),
            Ray Tune will sample new random parameters in each grid search
            condition.
        random_state:
            Seed or numpy random generator to use for reproducible results.
            If None (default), will use the global numpy random generator
            (``np.random``). Please note that full reproducibility cannot
            be guaranteed in a distributed environment.
```

Number of Trials

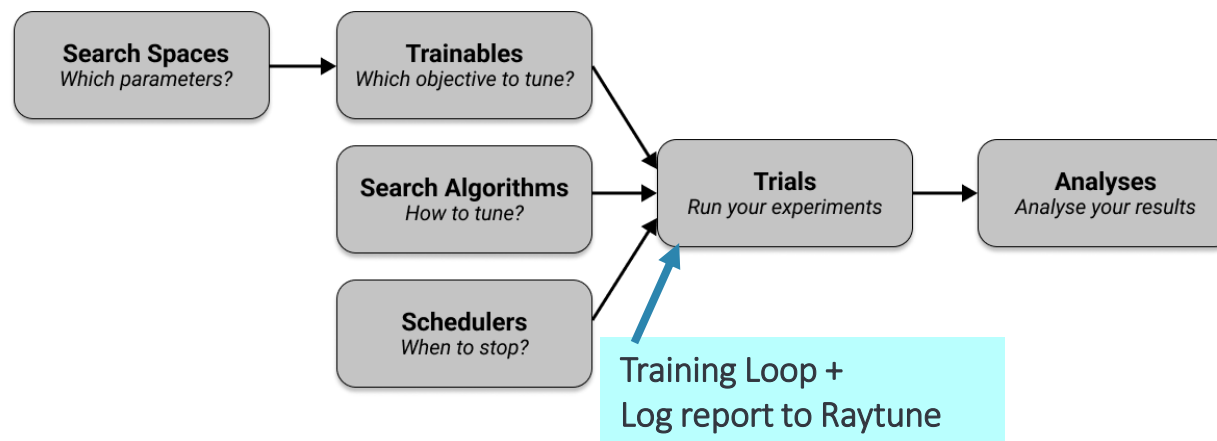
```
# Define the search space for hyperparameters
search_space = {
    "learning_rate": tune.grid_search([0.001, 0.01]), # Adjust learning
    "num_filters": tune.grid_search([32, 64]) # Adjust number of filter
}
```

Grid Search Parameter: $2 \times 2 = 4$ combinations -> 4 Trials

```
config = {
    'model_name': tune.grid_search(models_ls), 2
    'h_dims': tune.grid_search(h_dims), 2
    'optimizer': tune.grid_search(opts), 2
    'lr': tune.grid_search(lr), 2
    'batch_size': tune.grid_search(batch_size), 2
    'num_epochs': tune.grid_search(num_epochs), 2
    # 'num_epochs': tune.choice([1])
}
```

Grid Search Parameter: $2^6 = 64$ combinations -> 64 Trials

Raytune



1. Initialize Ray
2. define the search space:
3. define the trainable functions
4. specify the search algorithm

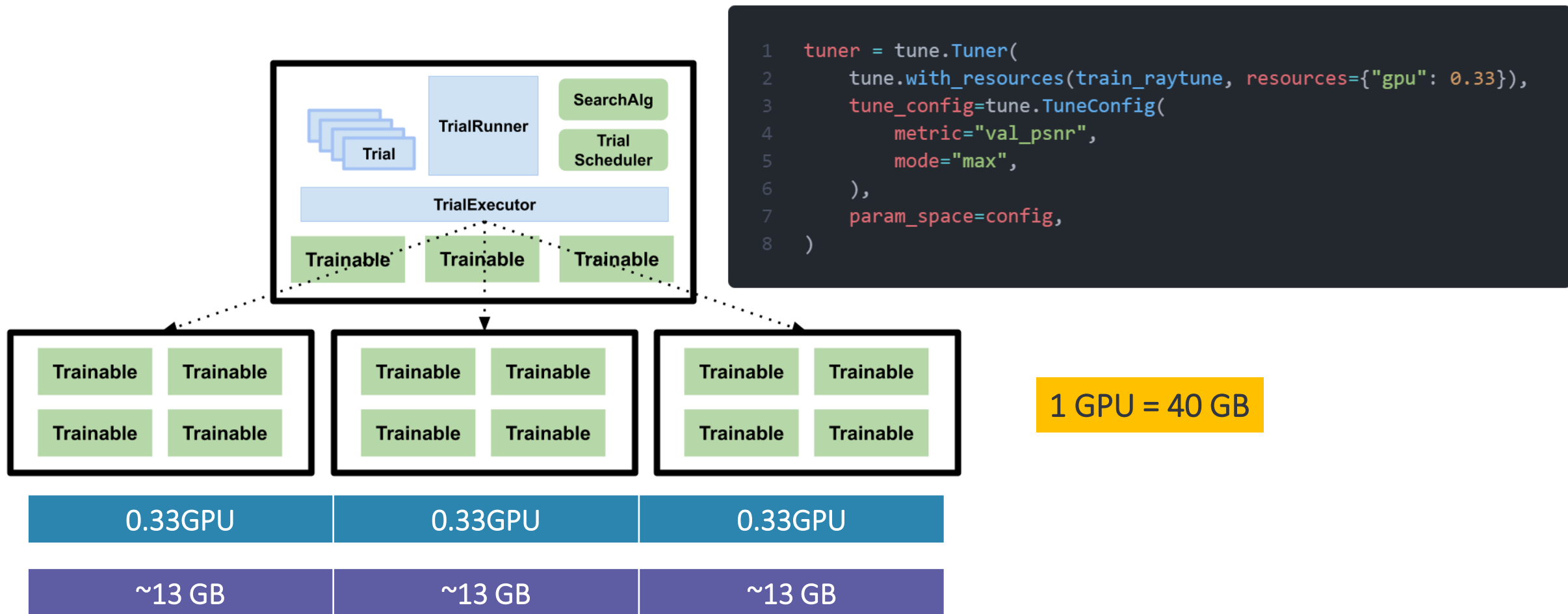
5. set up the scheduler and resources

6. Trials
7. Analyses

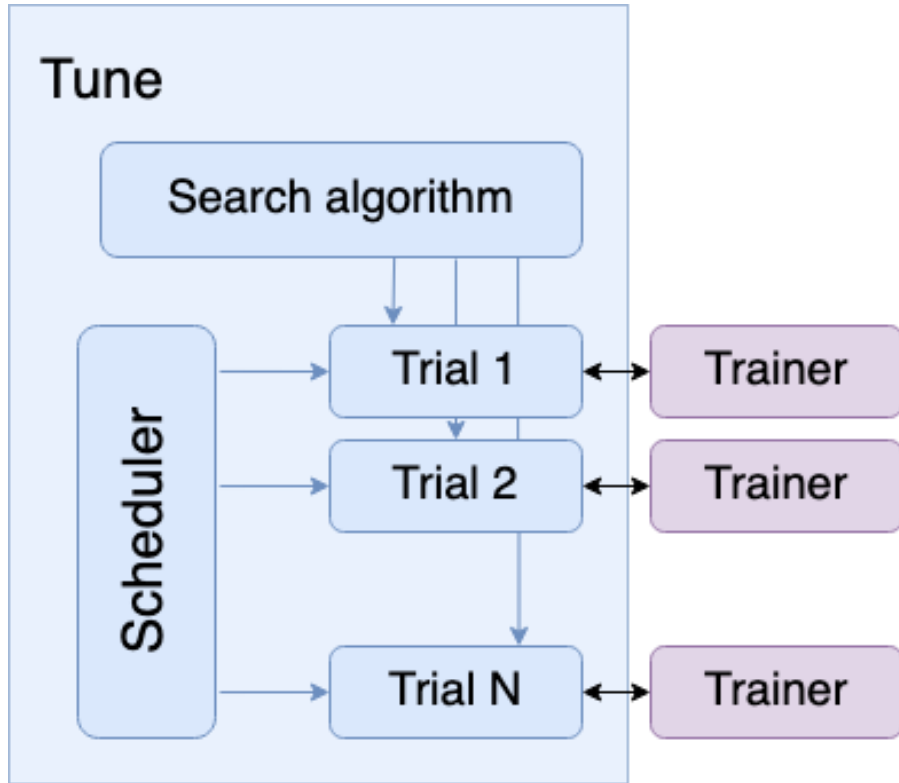
a Tune scheduler **can decide whether to stop the trial early or not**. If you don't specify a scheduler, Tune will use a **first-in-first-out (FIFO) scheduler by default**, which simply passes through the trials selected by your search algorithm in the order they were picked and does not perform any early stopping.

Scheduler Compatibility Matrix			
Scheduler	Need Checkpointing?	SearchAlg Compatible?	Example
ASHA	No	Yes	Link
Median Stopping Rule	No	Yes	Link
HyperBand	Yes	Yes	Link
BOHB	Yes	Only TuneBOHB	Link
Population Based Training	Yes	Not Compatible	Link
Population Based Bandits	Yes	Not Compatible	Basic Example, PPO example

Trial Scheduler (Fractional GPU)



Trial Scheduler (Fractional GPU)



```
1 tuner = tune.Tuner(  
2     tune.with_resources(train_raytune, resources={"gpu": 0.33}),  
3     tune_config=tune.TuneConfig(  
4         metric="val_psnr",  
5         mode="max",  
6     ),  
7     param_space=config,  
8 )
```

1 GPU x 2 = 40 x 2 GB

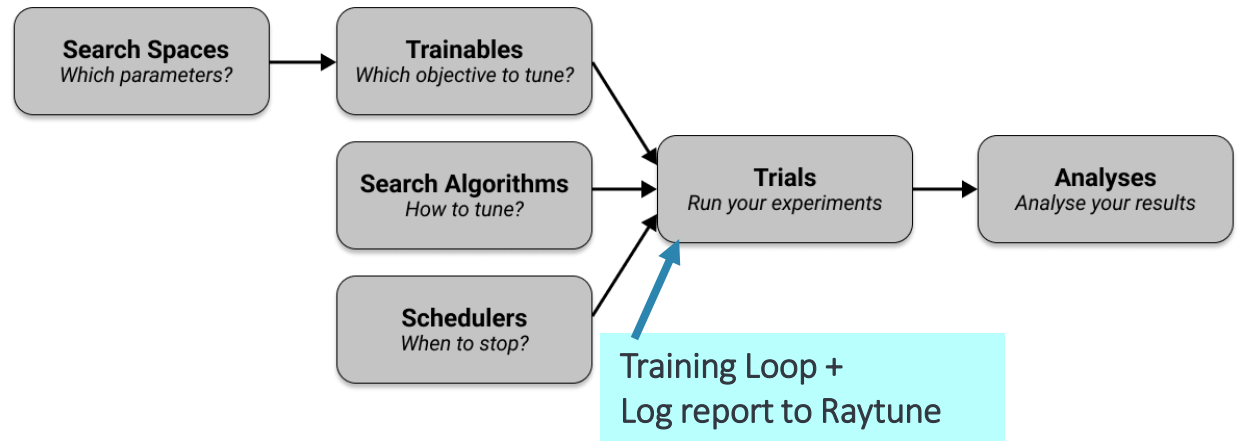
0.33GPU	0.33GPU	0.33GPU
~13 GB	~13 GB	~13 GB

1 GPU = 40 GB

0.33GPU	0.33GPU	0.33GPU
~13 GB	~13 GB	~13 GB

1 GPU = 40 GB

Raytune

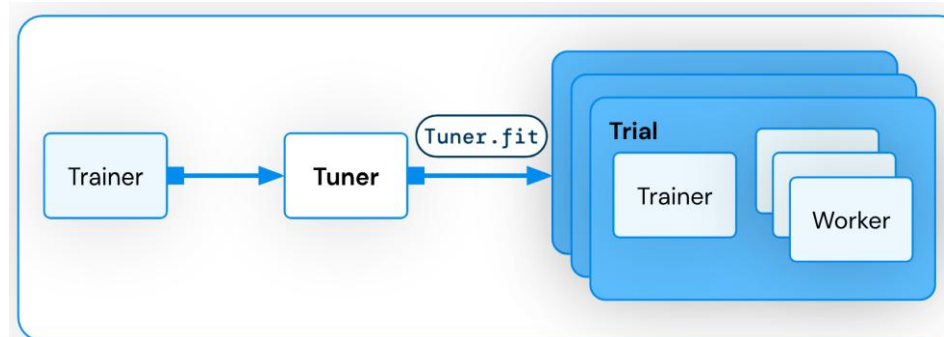


1. Initialize Ray
2. define the search space:
3. define the trainable functions
4. specify the search algorithm
5. set up the scheduler and resources
6. Run Trials
7. Analyses

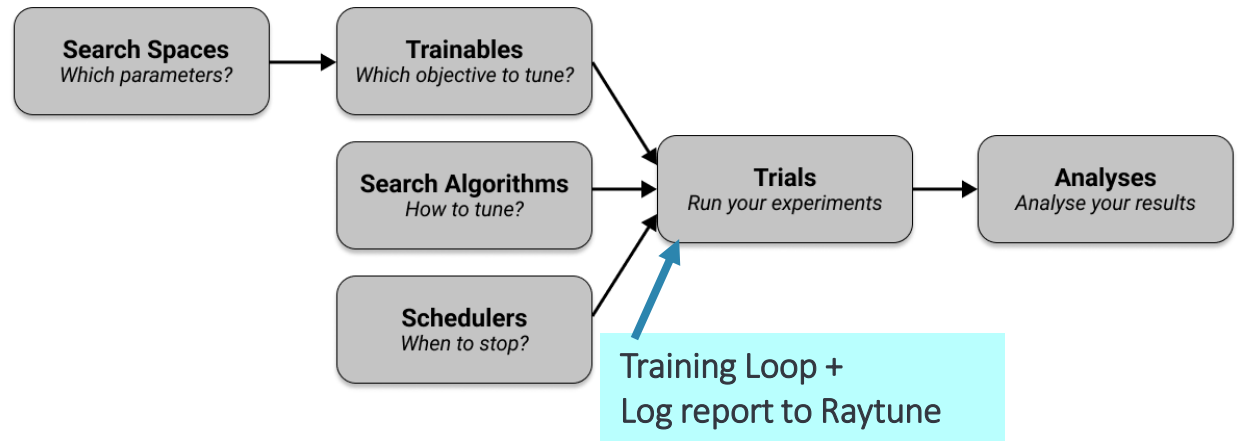
```
1 tuner = tune.Tuner(  
2     tune.with_resources(train_raytune, resources={"gpu": 0.5}),  
3     tune_config=tune.TuneConfig(  
4         metric="val_psnr",  
5         mode="max",  
6     ),  
7     param_space=config,  
8 )
```

Default Scheduler: FIFO

```
1 result = tuner.fit()
```



Raytune



1. Initialize Ray
2. define the search space:
3. define the trainable functions
4. specify the search algorithm
5. set up the scheduler and resources
6. Trials
7. Analyses

Sample Report from Raytune (only running .py)

Trial status: 4 RUNNING | 140 PENDING
Current time: 2024-09-12 21:58:51. Total running time: 48min 36s
Logical resource usage: 0/256 CPUs, 1.0/1 GPUs (0.0/1.0 accelerator_type:A100)
Current best trial: bb79a_00002 with val_psnr=19.168384967224043 and params={'architecture': [64, 128, 256, 512], 'optimizer': 'Adam', 'lr': 0.001, 'batch_size': 16, 'num_epochs': 10}

Trial name	status	architecture	optimizer	lr	batch_size	num_epochs	iter	total time (s)	train_loss	val_loss	val_psnr	val_ssim
train_raytune_bb79a_00000	RUNNING	[32, 64, 128]	Adam	0.001	16	10	8	2610.66	0.00464363	0.00424566	19.1575	0.478367
train_raytune_bb79a_00001	RUNNING	[64, 128, 256]	Adam	0.001	16	10	8	2615.27	0.00467182	0.00420267	19.1578	0.478375
train_raytune_bb79a_00002	RUNNING	[64, 128, 256, 512]	Adam	0.001	16	10	9	2807.03	0.00450838	0.00407761	19.1684	0.478713
train_raytune_bb79a_00003	RUNNING	[32, 64, 128]	Adam	0.001	32	10	9	2696.58	0.00454756	0.00407789	19.1679	0.47868
train_raytune_bb79a_00004	PENDING	[64, 128, 256]	Adam	0.001	32	10						
train_raytune_bb79a_00005	PENDING	[64, 128, 256, 512]	Adam	0.001	32	10						
train_raytune_bb79a_00006	PENDING	[32, 64, 128]	Adam	0.0008	16	10						
train_raytune_bb79a_00007	PENDING	[64, 128, 256]	Adam	0.0008	16	10						
train_raytune_bb79a_00008	PENDING	[64, 128, 256, 512]	Adam	0.0008	16	10						

135 more PENDING

Sample Report from Raytune (only running .ipynb)

Tune Status

Current time: 2024-09-11 13:37:44

Running for: 00:08:50.24

Memory: 8.4/15.5 GiB

System Info

Using FIFO scheduling algorithm.

Logical resource usage: 0/32 CPUs, 0.5/1 GPUs (0.0/1.0 accelerator_type:G)

Trial Status

Trial name	status	loc	batch_size	lr	num_epochs	optimizer	iter	total time (s)	train_loss	val_loss	val_psnr
train_raytune_1e150_00000	TERMINATED	172.20.124.27:21783	16	0.001	2	Adam	2	527.98	0.0096077	0.00776196	19.1704
train_raytune_1e150_00001	TERMINATED	172.20.124.27:21784	16	0.001	2	SGD	2	525.874	0.0232457	0.0202024	19.1705

Log Results

```
1 print("Best config is:", result.get_best_result().config)
2 print("Best result is:", result.get_best_result())
3 df = result.get_dataframe()
4 df.to_csv('result.csv', index=False)
```

```
🚀[INFO] Training is done!
Best config is: {'optimizer': 'SGD', 'lr': 0.001, 'batch_size': 16, 'num_epochs': 2}
Best result is: Result(
  metrics={'train_loss': 0.02324574246792646, 'val_loss': 0.020202363561673445, 'val_psnr': 19.170531128023335, 'val_ssim': 0.4798950263451028},
  path='/home/nouzen/ray_results/train_raytune_2024-09-11_13-28-54/train_raytune_1e150_00001_1_batch_size=16,lr=0.0010,num_epochs=2,optimizer=SGD_2024-09-11_13-28-54',
  filesystem='local',
  checkpoint=None
)
```

Result.csv

train_loss	val_loss	val_psnr	val_ssim	timestamp	checkpoint_dir_name	done	training_iteration	trial_id	date	time_this_iter_s	time_total_s
0.009607699	0.007761964	19.17041983	0.479865601	1726036664		FALSE	2	1e150_00000	2024-09-11_13-37-44	265.8116789	527.9799469
0.023245742	0.020202364	19.17053113	0.479895026	1726036662		FALSE	2	1e150_00001	2024-09-11_13-37-42	265.3006921	525.8737261

pid	hostname	node_ip	time_since_restore	iterations_since_restore	config/optimizer	config/lr	config/batch_size	config/num_epochs	logdir
21783	REGINLEIF	172.20.124.27	527.9799469	2	Adam	0.001	16	2	1e150_00000
21784	REGINLEIF	172.20.124.27	525.8737261	2	SGD	0.001	16	2	1e150_00001



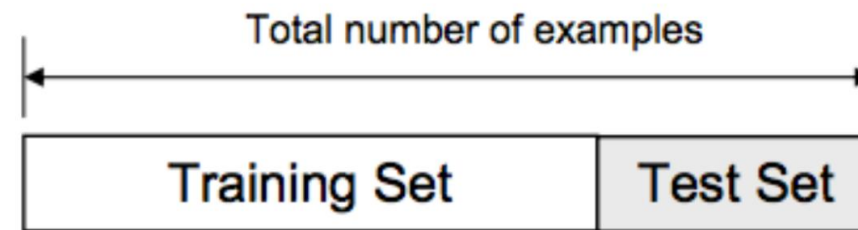
Model Validation

Train/test splitting

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3)
```

□ Quantity Split (ปริมาณ train/test data)

- Partition Dataset into
 - Model tolerance evaluation: XTest
 - Test_size = 0.3 = 30%
 - Model selection: using XTrain
 - Traing_size = 0.7 = 70% Training
 - Evaluate on various model learning parameters

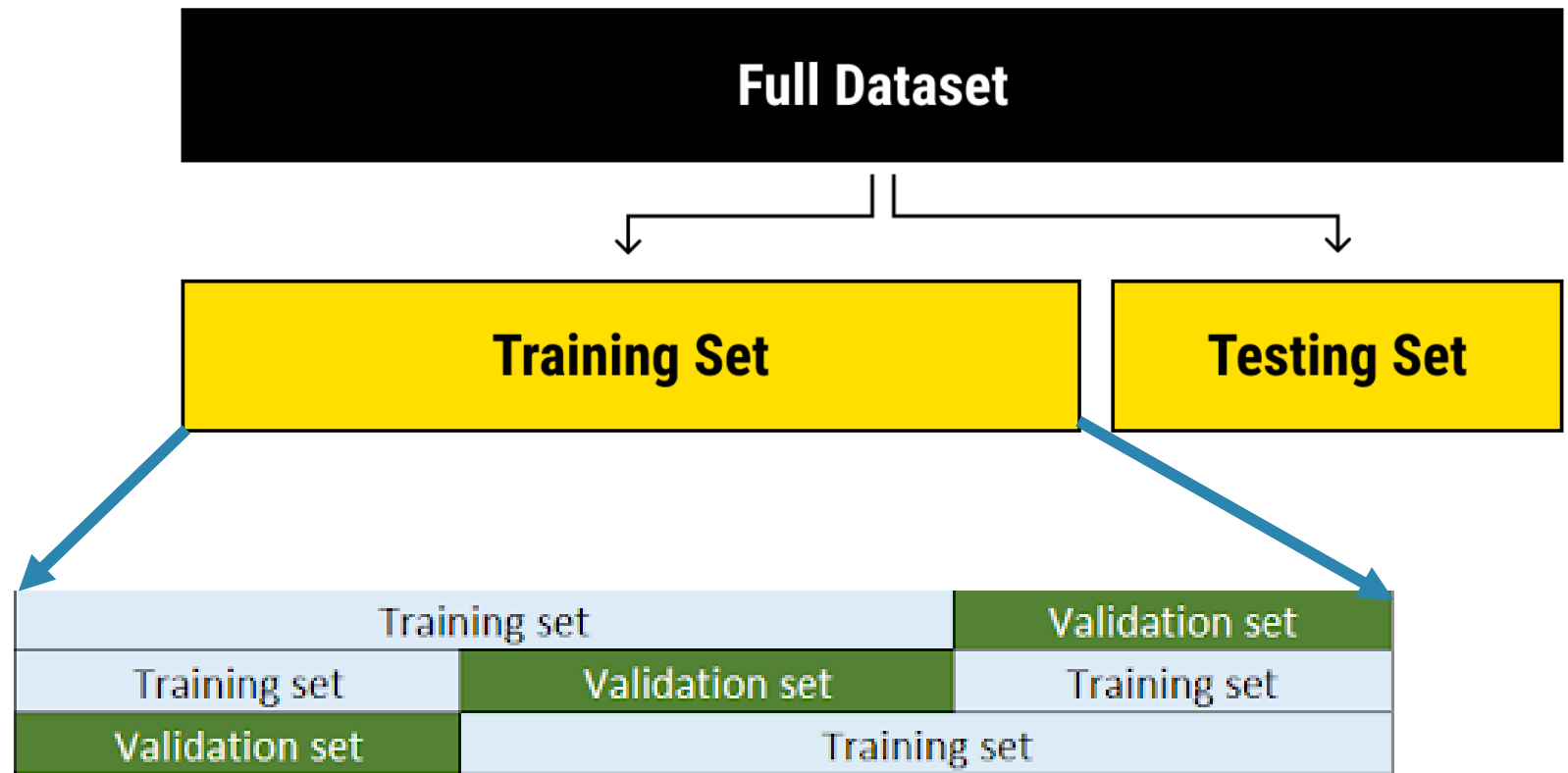


X_{train} for fitting model

X_{test} for prediction test

K-fold cv

- K-Fold Cross Validation
 - Partition Training into k subsets
 - For $i=1:k$ iteration
 - Select i th subset as test data
 - $k-1$ subsets are used to train



K-fold cv

```
1 def k_fold_cross_validation(model_class, dataset, optimizer_class, optimizer_params, num_folds=3, num_epochs=4, batch_size=16, device=None):
2
3
4     kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
5     fold_results = []
6
7     for fold, (train_idx, val_idx) in enumerate(kf.split(np.arange(len(dataset)))):
8         print(f'\n🔄 Fold {fold+1}/{num_folds}')
9         train_sampler = SubsetRandomSampler(train_idx)
10        valid_sampler = SubsetRandomSampler(val_idx)
11        train_loader = DataLoader(dataset, batch_size=batch_size, sampler=train_sampler)
12        valid_loader = DataLoader(dataset, batch_size=batch_size, sampler=valid_sampler)
13
14        model = model_class().to(device)
15        optimizer = optimizer_class(model.parameters(), **optimizer_params)
16        loss_fn = nn.CrossEntropyLoss()
17
18        for epoch in range(num_epochs):
19            model.train()
20
21            for images, labels in train_loader:
22                # Train model
23
24            avg_train_loss /= len(train_loader)
25            avg_train_acc /= len(train_loader)
26
27            model.eval()
28            with torch.no_grad():
29                for images, labels in valid_loader:
30                    # Evaluate model
31
32            avg_test_loss /= len(valid_loader)
33            avg_test_acc /= len(valid_loader)
34
35            print(f"Summary for Fold {fold+1} - Epoch [{epoch+1}/{num_epochs}]:")
36            print(f"\tTrain \tavg_loss: {avg_train_loss:.4f} \t| avg_acc: {avg_train_acc:.4f}")
37            print(f"\tTest \tavg_loss: {avg_test_loss:.4f} \t| avg_acc: {avg_test_acc:.4f}")
38
39            fold_results.append((avg_train_loss, avg_train_acc, avg_test_loss, avg_test_acc))
40
41        avg_train_loss = sum([result[0] for result in fold_results]) / num_folds
42        avg_train_acc = sum([result[1] for result in fold_results]) / num_folds
43        avg_test_loss = sum([result[2] for result in fold_results]) / num_folds
44        avg_test_acc = sum([result[3] for result in fold_results]) / num_folds
45
46        print("\n🎯 K-Fold Cross Validation Results:")
47        print(f"\tAverage Train Loss: {avg_train_loss:.4f}")
48        print(f"\tAverage Train Accuracy: {avg_train_acc:.4f}")
49        print(f"\tAverage Test Loss: {avg_test_loss:.4f}")
50        print(f"\tAverage Test Accuracy: {avg_test_acc:.4f}")
51
52    return fold_results, avg_train_loss, avg_train_acc, avg_test_loss, avg_test_acc
```


K-Fold cross validation

- ทำไมเราถึงต้องใช้ K-Fold Cross Validation
- K-Fold Cross Validation เป็นเทคนิคที่ใช้เพื่อประเมินประสิทธิภาพของโมเดลโดยการแบ่งชุดข้อมูลออกเป็นส่วนย่อยๆ (folds) หลายๆ ส่วน แล้ว train และ validate โมเดลหลายครั้ง ผลลัพธ์ที่ได้จะถูกวิเคราะห์ค่าสถิติเปรียบเทียบในแต่ละส่วนย่อยเพื่อดูความสม่ำเสมอของประสิทธิภาพโมเดลตลอดชุดข้อมูล
- วิธีการได้ข้อมูลทางสถิติจากแต่ละ Split (Fold)
 - 1. แบ่งชุดข้อมูลออกเป็น (K) ส่วนเท่าๆ กัน
 - 2. ทำการฝึกและทดสอบโมเดล (K) ครั้ง โดยในแต่ละครั้งจะใช้ส่วนที่ต่างกันเป็นชุด validate และส่วนที่เหลือเป็นชุด train
 - 3. เก็บค่าประสิทธิภาพของโมเดลจากแต่ละ fold เช่น Accuracy, Precision, Recall, F1-score
 - 4. คำนวณสถิติค่าเฉลี่ยและค่าเบี่ยงเบนมาตรฐานของค่าประสิทธิภาพที่ได้จากทุก fold เพื่อให้ได้ค่าประเมินที่รวมผลจากทุกส่วนของข้อมูล

K-Fold cross validation

- วิธีการตีความผลลัพธ์ทางสถิติจากแต่ละ Fold
 - - ค่าเฉลี่ย (Mean): ค่าประสิทธิภาพเฉลี่ยของโมเดลจากทุก fold ซึ่งบอกถึงประสิทธิภาพทั่วไปของโมเดล
 - - ส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation): ค่าที่บอกถึงความแปรปรวนของประสิทธิภาพของโมเดลจากทุก fold ยิ่งค่านี้น้อยยิ่งแสดงว่าโมเดลเรียนรู้ทุกส่วนของข้อมูลได้อย่างมีประสิทธิภาพ หากค่านี้มีค่ามาก หมายความว่า มีบางส่วนของข้อมูลที่โมเดลยังเรียนรู้ได้ไม่ดี อาจเป็นเพราะข้อมูลในชุด validate นั้น แตกต่างจากชุด train มาก และจำเป็นต้องนำออกมาวิเคราะห์ความผิดปกติของชุดข้อมูล และหาทางสอนโมเดลในกรณีผิดปกติในชุด validate นั้น เช่น เพิ่มการแก้ไขข้อมูลผิดปกติ ในขั้นตอน data correction เพื่อปรับเปลี่ยนหรือกำจัดทิ้ง หรือ สร้าง data augment จำลองชุด validate ที่มีปัญหานั้น หากเป็นข้อมูลผิดปกติที่ต้องการให้โมเดลเรียนรู้และเข้าใจ
 - - ค่าแยกเดี่ยว (Individual Fold Performance): ค่าประสิทธิภาพของแต่ละ fold ที่สามารถใช้เพื่อตรวจสอบว่ามี fold ใดที่มีผลการประเมินที่ผิดปกติหรือไม่
- การใช้ K-Fold Cross Validation เป็นเทคนิคที่มีประโยชน์มากในการประเมินความสม่ำเสมอของประสิทธิภาพของโมเดลตลอดชุดข้อมูล ช่วยให้เรามั่นใจได้ว่าโมเดลของเราสามารถทำงานได้ดีในสถานการณ์ที่แตกต่างกัน